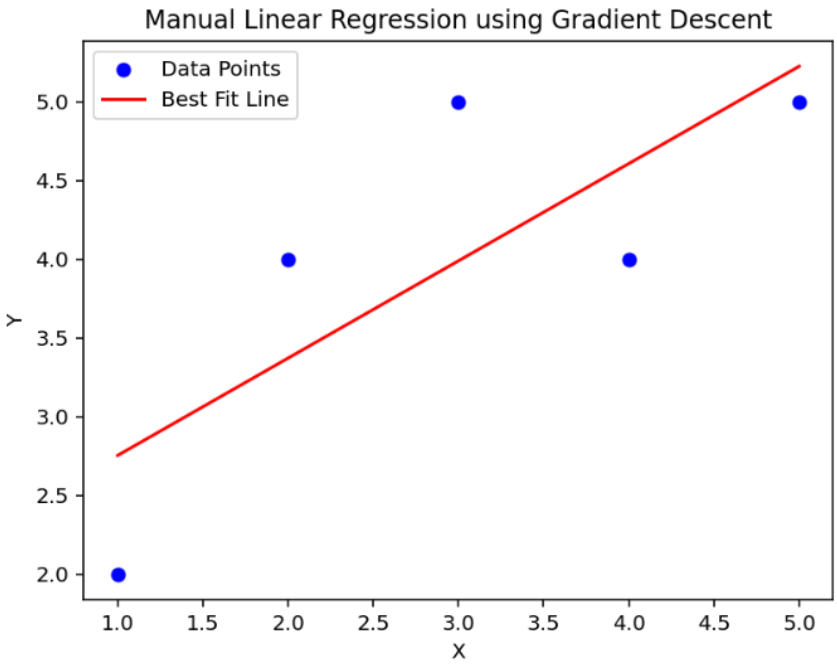
Assignment 5: Linear Regression Report

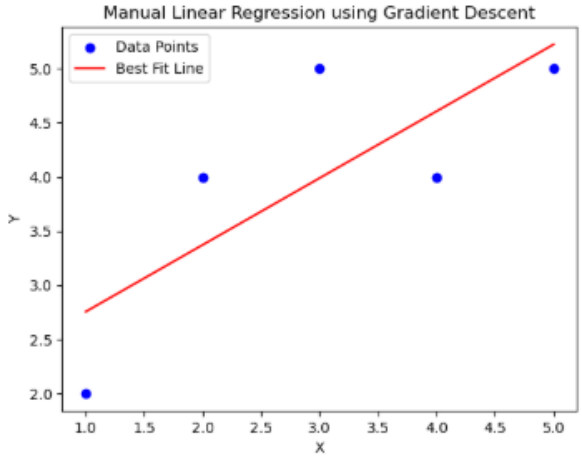
Student: Abhiney Kumar
Enrollment: 198409
ECE, NIT Jalandhar
Email: abhineyk.ec.24@nitj.ac.in

Exercise 1

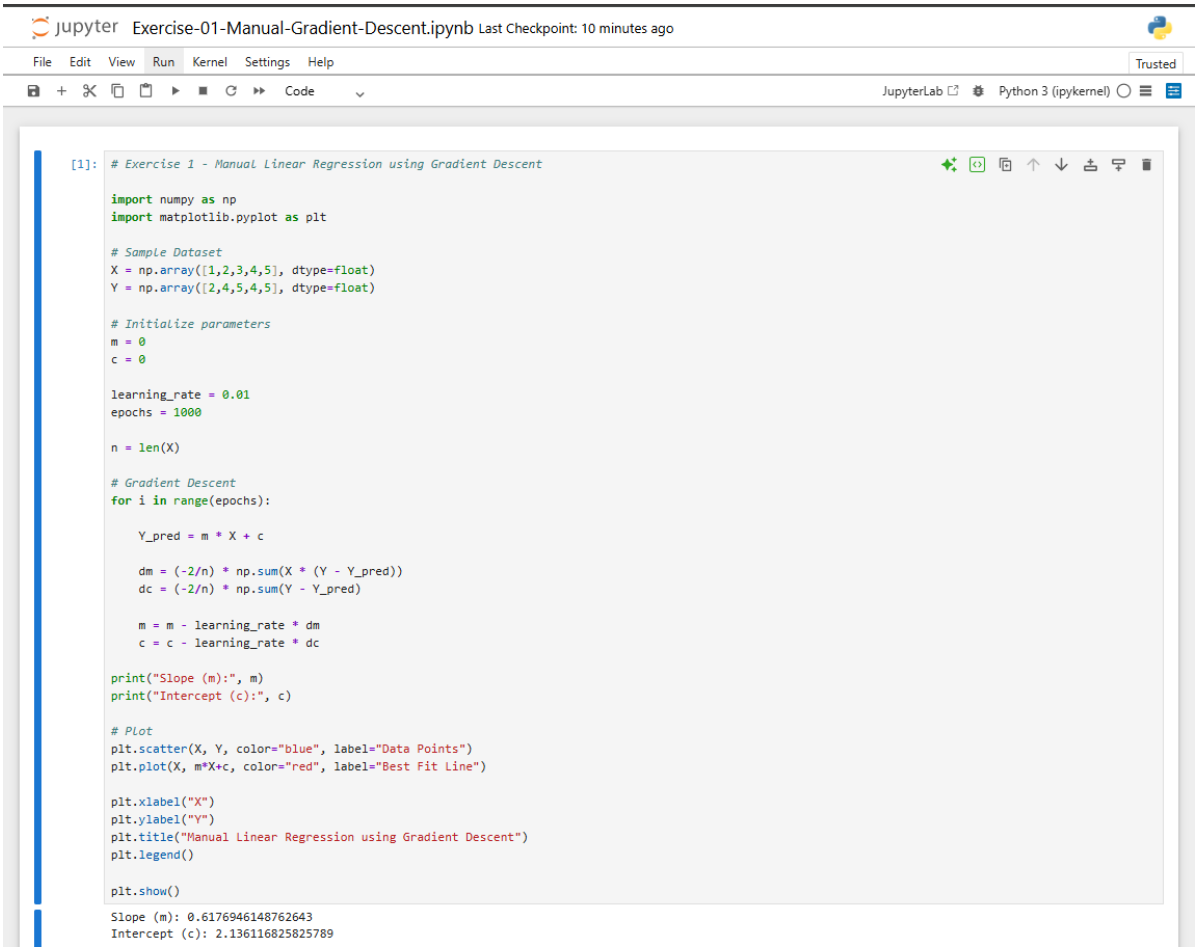


[1]:
```python
# Exercise 1 - Manual Linear Regression using Gradient Descent

import numpy as np
import matplotlib.pyplot as plt

# Sample Dataset
X = np.array([1,2,3,4,5], dtype=float)
Y = np.array([2,4,5,4,5], dtype=float)

# Initialize parameters
m = 0
c = 0

learning_rate = 0.01
epochs = 1000

n = len(X)

# Gradient Descent
for i in range(epochs):

    Y_pred = m * X + c

    dm = (-2/n) * np.sum(X * (Y - Y_pred))
    dc = (-2/n) * np.sum(Y - Y_pred)

    m = m - learning_rate * dm
    c = c - learning_rate * dc

print("Slope (m):", m)
print("Intercept (c):", c)

# Plot
plt.scatter(X, Y, color="blue", label="Data Points")
plt.plot(X, m*X+c, color="red", label="Best Fit Line")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Manual Linear Regression using Gradient Descent")
plt.legend()

plt.show()
```

```
Slope (m): 0.6176946148762643
Intercept (c): 2.136116825825789
```



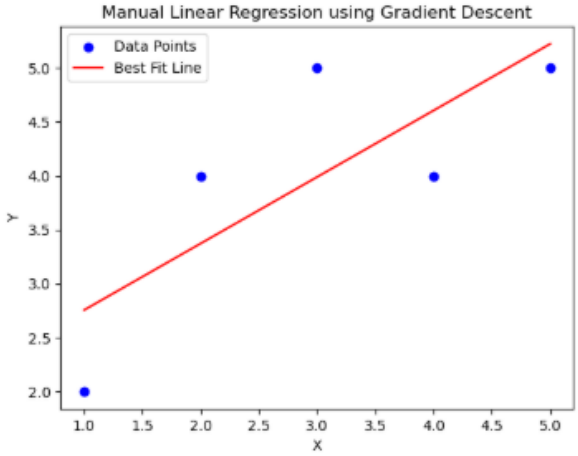
[ ]:


jupyter Exercise-01-Manual-Gradient-Descent.ipynb Last Checkpoint: 10 minutes ago
File Edit View Run Kernel Settings Help
Trusted
Code
JupyterLab
Python 3 (ipykernel)
[1]:
# Exercise 1 - Manual Linear Regression using Gradient Descent
import numpy as np
import matplotlib.pyplot as plt
# Sample Dataset
X = np.array([1,2,3,4,5], dtype=float)
Y = np.array([2,4,5,4,5], dtype=float)
# Initialize parameters
m = 0
c = 0
learning_rate = 0.01
epochs = 1000
n = len(X)
# Gradient Descent
for i in range(epochs):
Y_pred = m * X + c
dm = (-2/n) * np.sum(X * (Y - Y_pred))
dc = (-2/n) * np.sum(Y - Y_pred)
m = m - learning_rate * dm
c = c - learning_rate * dc
print("Slope (m):", m)
print("Intercept (c):", c)
# Plot
plt.scatter(X, Y, color="blue", label="Data Points")
plt.plot(X, m*X+c, color="red", label="Best Fit Line")
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Manual Linear Regression using Gradient Descent")
plt.legend()
plt.show()
Slope (m): 0.6176946148762643
Intercept (c): 2.136116825825789

Exercise 2

[1]:
```python
# Exercise 1 - Manual Linear Regression using Gradient Descent

import numpy as np
import matplotlib.pyplot as plt

# Sample Dataset
X = np.array([1,2,3,4,5], dtype=float)
Y = np.array([2,4,5,4,5], dtype=float)

# Initialize parameters
m = 0
c = 0

learning_rate = 0.01
epochs = 1000

n = len(X)

# Gradient Descent
for i in range(epochs):

    Y_pred = m * X + c

    dm = (-2/n) * np.sum(X * (Y - Y_pred))
    dc = (-2/n) * np.sum(Y - Y_pred)

    m = m - learning_rate * dm
    c = c - learning_rate * dc

print("Slope (m):", m)
print("Intercept (c):", c)

# Plot
plt.scatter(X, Y, color="blue", label="Data Points")
plt.plot(X, m*X+c, color="red", label="Best Fit Line")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Manual Linear Regression using Gradient Descent")
plt.legend()

plt.show()
```

```
Slope (m): 0.6176946148762643
Intercept (c): 2.136116825825789
```



Exercise 3

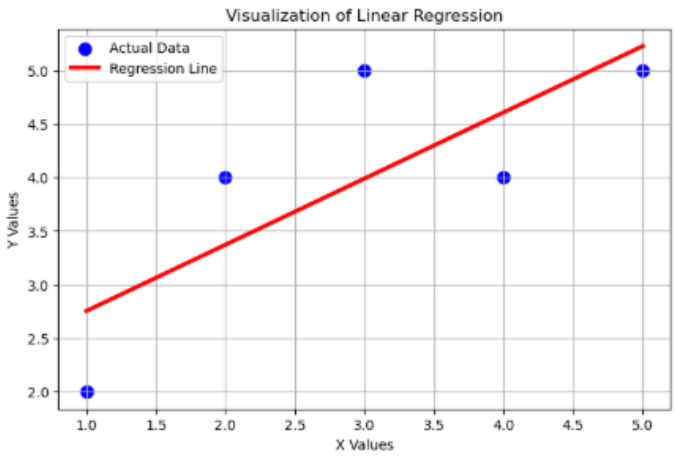
jupyter Exercise-02-Regression-Visualization.ipynb Last Checkpoint: 1 minute ago

File Edit View Run Kernel Settings Help Trusted

Code JupyterLab Python 3 (ipykernel)

[1]:

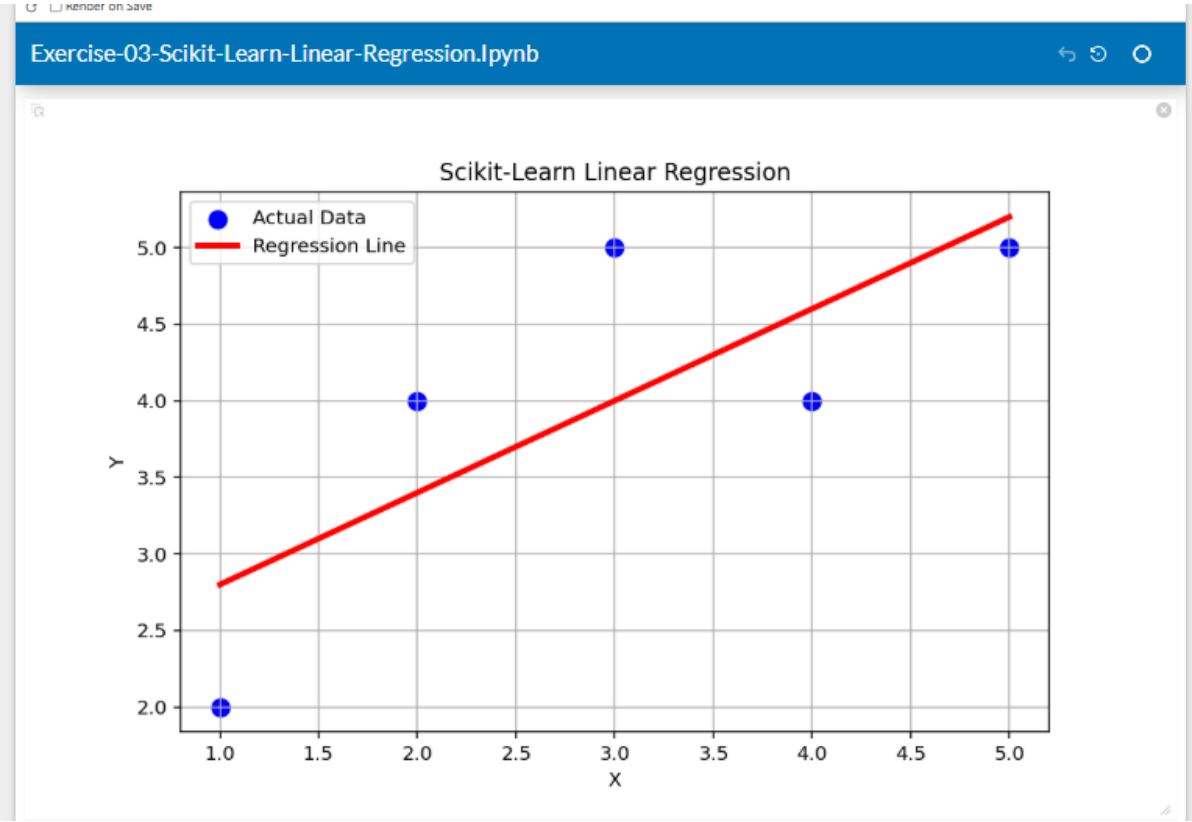
```
# Exercise 2 - Linear Regression Visualization

import numpy as np
import matplotlib.pyplot as plt

# Sample Data
X = np.array([1,2,3,4,5])
Y = np.array([2,4,5,4,5])

# Line of Best Fit
m = 0.6176946148762643
c = 2.136116825825789

Y_pred = m * X + c

plt.figure(figsize=(8,5))

plt.scatter(X, Y,
            color="blue",
            s=80,
            label="Actual Data")

plt.plot(X,
         Y_pred,
         color="red",
         linewidth=3,
         label="Regression Line")

plt.xlabel("X Values")
plt.ylabel("Y Values")
plt.title("Visualization of Linear Regression")
plt.grid(True)
plt.legend()

plt.show()
```



[ ]:

1]:
```python
# Exercise 3 - Linear Regression using Scikit-Learn

import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Sample Dataset
X = np.array([1,2,3,4,5]).reshape(-1,1)
Y = np.array([2,4,5,4,5])

# Create Model
model = LinearRegression()

# Train Model
model.fit(X, Y)

# Predictions
Y_pred = model.predict(X)

# Results
print("Slope (Coefficient):", model.coef_[0])
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mean_squared_error(Y, Y_pred))
print("R² Score:", r2_score(Y, Y_pred))

# Predict new value
new_x = np.array([[6]])
prediction = model.predict(new_x)

print("Prediction for X = 6 :", prediction[0])

# Plot
plt.figure(figsize=(8,5))

plt.scatter(X, Y,
            color="blue",
            s=80,
            label="Actual Data")

plt.plot(X,
         Y_pred,
         color="red",
         linewidth=3,
         label="Regression Line")

plt.xlabel("X")
plt.ylabel("Y")
plt.title("Scikit-Learn Linear Regression")
plt.grid(True)
plt.legend()

plt.show()
```

```
Slope (Coefficient): 0.6000000000000002
Intercept: 2.1999999999999993
Mean Squared Error: 0.4799999999999998
R² Score: 0.6000000000000001
Prediction for X = 6 : 5.800000000000001
```



Conclusion

All Assignment 5 exercises were completed successfully.